# 30 多項式與快速傅立葉轉換

*Part VII Selected Topics*

## 本章定位

多項式乘法藉相異點求值與插值轉成逐點乘法；本章從單位根正交性證明DFT與inverse，再逐層核對偶奇分解、bit-reversal、蝶形索引與補零，連結精確算術和浮點限制。

## 前置知識

複數根、點值表示與遞迴分治。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 本章路線

*學習目標*

- 比較係數與點值表示的乘法成本。
- 推導 DFT 與逆 DFT。
- 實作 radix-2 FFT 並理解單位根條件。
- 30.1 Representing polynomials
- 30.2 The DFT and FFT
- 30.3 Efficient FFT implementations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 30.1 Representing polynomials

係數表示$A(x)=\Sigma_{j=0}^{n-1}a_jx^j$的degree bound是n，也就是degree<n；最高係數可以為0，實際degree可能更小。零多項式在程式可用單一係數[0]表示，不把degree bound與向量中第一個非零位置混淆。以下係數依**低次到高次**排列。

同一degree bound的加法逐係數相加，$\Theta(n)$。在一點z求值可用Horner：先$v=a_{n-1}$，再對$j=n-2,\cdots,0$做$v=a_j+zv$，每次保持$v=\Sigma_{r=j}^{n-1}a_rz^{r-j}$，最後得到A(z)，共n−1次乘與n−1次加。任選n個點分別做Horner，則$\Theta(n^2)$，不是$\Theta(n)$。

兩個長度r、s係數陣列相乘，$c_k=\Sigma_{i+j=k}a_ib_j$，輸出長r+s−1；直接卷積有rs次係數乘法，$\Theta(rs)$。FFT將改善的是整個表示轉換加乘法流程，不只是中間的pointwise product。



## 相異點值的存在性與唯一性

給n個相異$x_0,\cdots,x_{n-1}$及值$y_0,\cdots,y_{n-1}$。Lagrange公式$A(x)=\Sigma_k y_k\ell_k(x)$，其中$\ell_k(x)=\Pi_{j\neq k}(x-x_j)/(x_k-x_j)$，每個分母非零、degree<n，且$\ell_k(x_i)$只有i=k時為1，其餘為0，故插值多項式存在。

若兩個degree<n多項式同時吻合，差H有n個不同根。非零degree d多項式至多d根：遇根α可依因式定理寫$H=(x-\alpha)Q$，其餘不同根亦為Q的根，對degree歸納即可。因此H必為零，唯一性成立。少於n點時，可在一個解上加任意常數倍Π已知點$(x-x_j)$，仍吻合，故一般不唯一。

矩陣寫法是$V[i,j]=x_i^j$、$Va=y$，V稱Vandermonde matrix。其$\det=\Pi_{j<k}(x_k-x_j)$：兩個x相同會有相同row，故每個差都是det多項式的因子；總degree正好$n(n-1)/2$，對照單項$x_1x_2^2\cdots x_{n-1}^{n-1}$的係數1便確定常數。因此相異點讓V可逆，也可用一般線性系統求插值，但直接LUP是$\Theta(n^3)$。

公式也可在$O(n^2)$算術操作內轉成係數：先逐因子建$Q(x)=\Pi_j(x-x_j)$，每步更新總$O(n^2)$；對每個k用synthetic division在線性時間求$Q/(x-x_k)$，分母各用O(n)算，再逐係數加總。若每次從頭重建每個$\ell_k$的多項式，則不能直接宣稱相同的二次成本。

## 點值相乘需要足夠長度

在**相同且相異的N個點**存A、B的值，$C(x_k)=A(x_k)B(x_k)$只需N次乘法。要唯一重建乘積，須$N \geq r+s-1$，並將兩個係數陣列都補零到N；radix-2選不小於該值的2冪。

例如$A=1+2x$、$B=1-x$，$r=s=2$，乘積$1+x-2x^2$需3係數，所以選$N=4$。不能只因兩個輸入都長2就直接做2點轉換。

若在N次單位根只留N點，$x^N=1$，所以插值得到的是模$x^N-1$的循環卷積：位置k收到$c_k+c_{k+N}+c_{k+2N}+\cdots$，高次項會繞回。例$(1+x)^2=1+2x+x^2$，$N=2$時$x^2$與常數無法區分，反轉只得[2,2]；[1,2,1]才是普通乘積。補零不是可有可無的格式處理。



## 30.2 The DFT and FFT

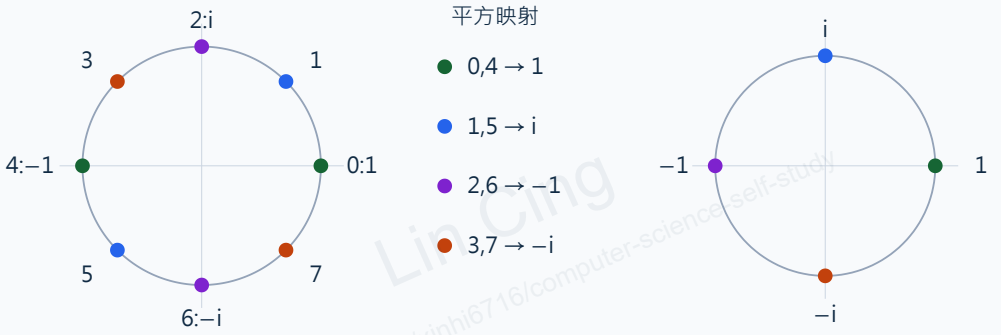
本章固定**正指數forward**：$\omega_n=e^{2\pi i/n}$，$y_k=\sum_{j=0}^{n-1}a_j\omega_n^{jk}=A(\omega_n^k)$。其他領域常把forward指數設負；只要正逆成對就成立，但不能混用兩種慣例。

$\omega_n$是primitive root，亦即$\omega_n^k=1$當且僅當n整除k。其n個冪相異，構成合法插值點。對正整數d有$\omega_{dn}^{dk}=e^{2\pi ik/n}=\omega_n^k$（cancellation）。n偶數時$\omega_n^{n/2}=-1$，且$(\omega_n^k)^2=\omega_{n/2}^k$；k與k+n/2的平方相同，全部n個點平方後只剩n/2個不同點，各出現兩次（halving）。

幾何級數給正交性：$\sum_{j=0}^{n-1}\omega_n^{jt}$在$n \mid t$時每項1、和n；否則$\omega_n^t\neq 1$，和$=(1-(\omega_n^t)^n)/(1-\omega_n^t)=0$。分母是否非零正是primitive條件要處理的地方，不能把ω=1當成任意n的替代root。

**八個單位根平方後，恰剩四個不同點**

左標k表示$\omega_8^k$；同色相對兩點平方後，落到右側同色的一點。




Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 正交性推導 Inverse DFT

定義$F[k,j]=\omega_n^{jk}$。令$G[j,k]=(1/n)\omega_n^{-jk}$，則$(GF)[j,\ell]=(1/n)\Sigma_k\omega_n^{k(l-j)}$。j=ℓ時為1，其餘因$0\le j,\ell<n$而n不整除ℓ−j，為0，所以GF=I，G就是$F^{-1}$。

因此$a_j=(1/n)\Sigma_k y_k\omega_n^{-jk}$：使用負指數twiddle，**最後除n一次**。也可每層分攤正規化，但不能每層除n，更不能完全漏除。

若錯把forward再做一次後除n，利用同一和式得到$F^2a/n$的第j項是a[(-j) mod n]，是循環索引反轉；對任意複數輸入不等於「取共軛」。n=4、a=[2,5+i,7,−1]會變[2,−1,7,5+i]，可直接分辨兩種錯法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 偶奇拆解的兩個子問題

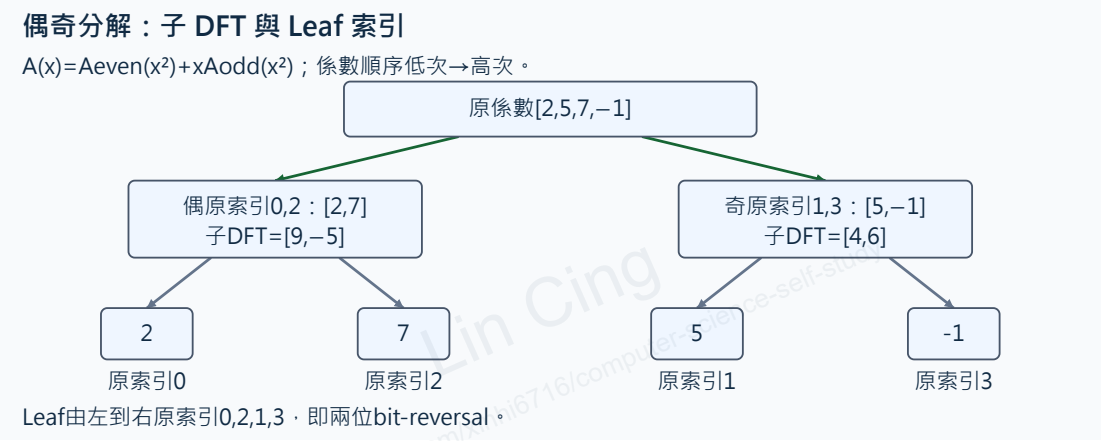
設n是正2冪，$A_e(z)=a_0+a_2z+\cdots$、$A_o(z)=a_1+a_3z+\cdots$，則$A(x)=A_e(x^2)+xA_o(x^2)$。由halving lemma，在全部n個root求A只需兩個n/2點DFT E、O。

對$0\le k<n/2$，$y_k=E_k+\omega_n^k O_k$，$y_{k+n/2}=E_k-\omega_n^k O_k$，因$\omega_n^{k+n/2}=-\omega_n^k$。一次算$t=\omega_n^k O_k$，使用加、減產生兩個輸出，就是butterfly。遞迴基例n=1直接回$[a_0]$；歸納子轉換正確，再用等式合併全部n個值，證明整個FFT正確。

取a=[2,5,7,−1]。偶係數[2,7]的2點DFT為[9,−5]；奇係數[5,−1]為[4,6]。k=0用twiddle1，得$y_0=13$、$y_2=5$；k=1用i，得$y_1=-5+6i$、$y_3=-5-6i$。四個中間值彼此不同，方便分辨接線是否真正正確。



## 偶奇拆解的兩個子問題 （續）



## 遞迴算法與完整成本

```
FFT(a,sign):
  n=len(a)；要求n為正2冪
  if n==1: return [a[0]]
  E=FFT([a[0],a[2],...], sign)
  O=FFT([a[1],a[3],...], sign)
  w=1; wm=exp(sign*2πi/n)
  for k=0..n/2-1:
    t=w*O[k]
    y[k]=E[k]+t; y[k+n/2]=E[k]-t
    w=w*wm
  return y
```

forward取sign=+1；inverse取−1，最外層回來後除n。每層Θ(n)拆分/合併，T(n)=2T(n/2)+Θ(n)，$\log_2 n$層，Θ(n log n)複數算術。n/2 butterflies每層，共$(n/2)\log_2 n$個。

複製偶奇陣列且及時釋放子結果的順序實作，可用O(n)峰值額外空間與O(log n)stack；若保存所有層的陣列歷史則O(n log n)。不能只由遞迴深度就宣稱全部記憶體O(log n)，因每層還有輸出與子陣列。



## 卷積的 Forward、點乘、Inverse 全部算值

A=[1,2,0,0]、B=[1,−1,0,0]，N=4、$\omega_4=i$。

| k | DFT(A) | DFT(B) | 逐點乘積 |
|---|---|---|---|
| 0 | 3 | 0 | 0 |
| 1 | 1+2i | 1−i | 3+i |
| 2 | −1 | 2 | −2 |
| 3 | 1−2i | 1+i | 3−i |

對[0,3+i,−2,3−i]做負號DFT，未除4的結果[4,4,−8,0]；除4得[1,1,−2,0]，截取前3項即$1+x-2x^2$。兩次forward加一次inverse與Θ(N)點乘，總Θ(N log N)，輸出degree bound不足N的條件使循環與普通卷積一致。

## 卷積的 Forward、點乘、Inverse 全部算值 (續)

### 一次卷積的全部頻點與反插值

A=1+2x，B=1−x，均補到N=4；$\omega_4=i$，forward取正指數。

| k | DFT(A) | DFT(B) | 逐點乘積 |
|---|---|---|---|
| 0 | 3 | 0 | 0 |
| 1 | 1+2i | 1−i | 3+i |
| 2 | −1 | 2 | −2 |
| 3 | 1−2i | 1+i | 3−i |

負號DFT先得[4,4,−8,0]，除4→[1,1,−2,0]。

綠格 = 乘積點值；取前3個係數，得到$1+x-2x^2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 30.3 Efficient FFT implementations

遞迴每層先取偶索引再取奇索引，也就是先看原索引最低位，再看下一位。若$n=2^b$，從左到右的leaf順序就是把原索引b個bits反轉後的順序。rev是自身的逆，因此可寫A[rev(j)]=a[j]，也可寫A[j]=a[rev(j)]。

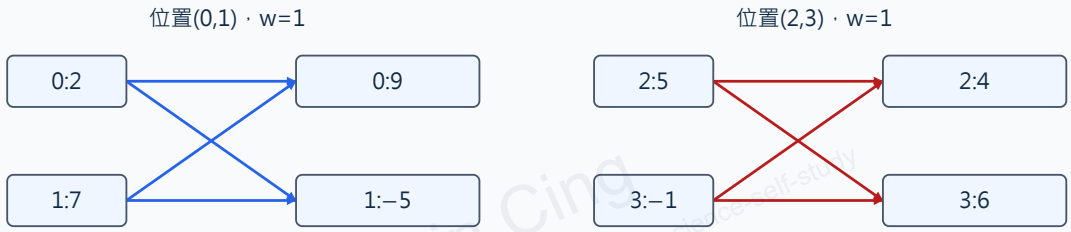
n=4：原索引00,01,10,11反轉為00,10,01,11，輸入[2,5,7,−1]重排為[2,7,5,−1]。重排後stage1配對的是**實體位置(0,1)、(2,3)**，分別含原索引(0,2)、(1,3)；不能把「原索引」與「目前陣列位置」寫成同一個欄位。

stage1兩個twiddle皆1，實體陣列變[9,−5,4,6]。stage2的m=4，配對實體位置(0,2)與(1,3)，依序使用1與i，輸出按位置0,1,2,3是[13,−5+6i,5,−5−6i]。stage2若又配相鄰位置就是另一張圖、另一個運算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

### Stage1：重排後合併相鄰位置

格內 = 實體位置:數值；上輸出u+wv，下輸出u−wv。




左藍pair含原索引(0,2)；右紅pair含原索引(1,3)。

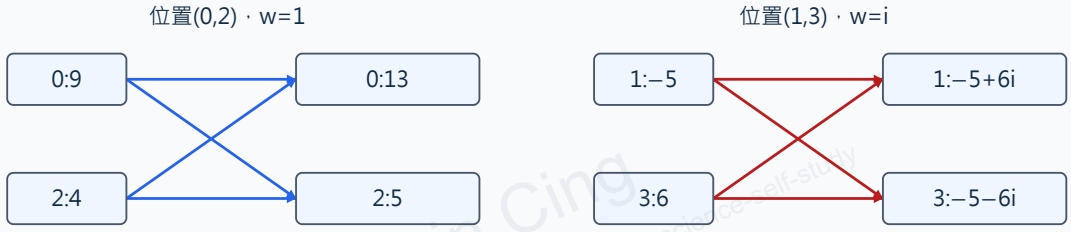
m=2後實體陣列：[9,−5,4,6]；下一stage間距改為2。



## Stage2 必須跨半塊合併

### Stage2：跨半塊位置與真 Twiddle

格內 = 實體位置:數值；上輸出u+wv，下輸出u−wv。




藍pair為(0,2)：9±4；紅pair為(1,3)：−5±i·6。

m=4後按位置0,1,2,3輸出：[13,−5+6i,5,−5−6i]。

| j | u=A[j] | t=ω₄ʲA[j+2] | 新A[j] | 新A[j+2] |
|---|---|---|---|---|
| 0 | 9 | 1 • 4=4 | 13 | 5 |
| 1 | −5 | i • 6=6i | −5+6i | −5−6i |

## Stage2 必須跨半塊合併 （續）

另一組a=[1,2,0,−1]依同一規則：bit-reversal→[1,0,2,−1]，stage1→[1,1,1,3]，stage2→[2,1+3i,0,1−3i]。這個例子有多個相同1，單看最終數字可能掩蓋接線錯誤；上面的不同中間值讓每條運算的來源更清楚。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Iterative FFT 的實體索引契約

```
將a按bit-reversal放入A
for s=1..log₂n:
  m=2^s; wm=exp(sign*2πi/m)
  for base=0,m,2m,...,n−m:
    w=1
    for j=0..m/2−1:
      u=A[base+j]; t=w*A[base+j+m/2]
      A[base+j]=u+t
      A[base+j+m/2]=u−t
      w=w*wm
inverse時最後每項除n
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每輪s開始，長度m區塊的左右半塊各已是相應偶／奇子問題的DFT；butterfly做與遞迴相同的合併，輪末每塊為完整m點DFT。s=1時半塊為單點，基例成立；最後m=n完成整體。先保存u再覆寫上格，否則下格的u−t會誤讀已更新值。

每stage恰n/2個butterflies，共Θ(n log n)。逐索引反轉b位元的簡單重排需O(n log n)位元步驟；可用額外n長陣列，或利用rev的2-cycle就地交換。twiddle只依m與j，不依base，同stage各block可共用其表，若存所有stage表總Θ(n)項；這不改變主要算術級序。

## Parallel FFT Circuit 的 Work 與 Depth

固定twiddle並把bit-reversal視為接線的bounded-fan-in電路，每stage n/2個互不衝突butterflies可同時做；每stage的輸出只有下一stage才讀。n=8有3stage、共12個butterflies，電路work $\Theta(n \log n)$、depth $\Theta(\log n)$，因每條輸入到輸出的路徑穿過log n次合併。

這個depth計算假設一次複數乘、加各為常數層，且沒有把twiddle串行生成、處理器通訊或實體wire長度藏在其中。軟體多執行緒實作若共用陣列，須在每stage結束同步；同stage各butterfly寫不同兩格，下一stage讀前一stage結果，才能使用相同DAG分析。若僅有P處理器，work下界W/P仍在，不會因電路畫成log n層就自動只需log n時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 精確算術與浮點 FFT 的界線

$\Theta(n \log n)$是複數算術操作模型；浮點root與每次乘加都有誤差。整數係數卷積最後四捨五入要有「每格絕對誤差<1/2」的保證，不能只因少數round-trip吻合就保證任意長度／大係數都精確。

一種精確替代是在質數域$F_p$用NTT：要求$N \mid (p-1)$，選order恰為N的$\omega$，則N在域中可逆，幾何和與inverse proof仍成立。**複合模數上僅宣稱primitive並不自動保證所需分母都是unit**，這裡的敘述限定質數域。

同一$A=[1,2,0,0]$、$B=[1,-1,0,0]$，$p=17$、$N=4$、$\omega=13$，$\omega^2=16\neq 1$、$\omega^4=1$，且$4^{-1}=13$。$NTT(A)=[3,10,16,9]$，$NTT(B)=[0,5,2,14]$，點乘mod17得$[0,16,15,7]$，inverse得$[1,1,15,0]$。把15解碼為$-2$，還原原乘積。

解碼也需要幅度界：若輸入整數係數$abs \le B$，長度r,s，每格卷積$abs \le \min(r,s)B^2$；取$p>2\min(r,s)B^2$才可從單模數唯一還原有號係數。本例$r=s=2$、$B=2$，p17>16足夠。單有模數正確結果，不代表任意大整數的原值沒有繞回。

NTT的FFT級序以mod p算術為單位；若p隨資料幅度增長，位元成本還需計入log p位元整數的乘加，不能把任意長整數乘法當作一次固定時間操作。

## 資料來源

*書目*

## 章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 30。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study